1. Basic Linux Commands (Part 1)
Here are the commands you'll be using:

a) mkdir – Make directories
Usage: mkdir <directory_name>

Example:
mkdir test_directory
This will create a new directory named test_directory in your current location.

b) chdir – Change directory
Usage: cd <directory_name>

Example:

cd test_directory
This will take you inside the test_directory you just created.

c) cat – Display the contents of a file
Usage: cat <file_name>

Example:

cat sample.txt
This will display the contents of the sample.txt file. (If the file doesn't exist, it will show an error.)

d) ls – List files and directories
Usage: ls

Example:

ls
This will list all files and directories in your current working directory. If you want more detailed info, use ls -l to show file permissions, ownership, and size.

e) chmod – Change file permissions
Usage: chmod <permissions> <file_name>

Example:

chmod 755 test_file.sh
This will set read, write, and execute permissions for the file test_file.sh for the owner, and read and execute permissions for others.

Steps to Follow:
Open a new bash terminal in Replit.

Run the commands mentioned above one by one, and experiment with creating directories, changing directories, listing files, and modifying file permissions.

For chmod, create a file (e.g., test_file.sh) and modify its permissions using chmod to see the effects.

how to write in a file

ChatGPT said:
To write in a file in the bash environment on Replit (or any other Linux terminal), you can use several methods. Here are some of the most common ways:

1. Using echo Command
You can use the echo command to write text into a file. Here's how:

Basic Syntax:

echo "Your text here" > filename.txt
This will create a new file filename.txt if it doesn't already exist and write the text "Your text here" into it. If the file already exists, it will overwrite the contents.

Example:

echo "Hello, this is a test file" > myfile.txt
This will create (or overwrite) the file myfile.txt with the text Hello, this is a test file.

Appending to a File:
If you want to add content to a file without overwriting it, use >> instead of >:

echo "Appending more text" >> myfile.txt
2. Using cat Command
You can also use the cat command to write multiple lines to a file.

Basic Syntax:

cat > filename.txt
After running this command, you can start typing. When you're done, press Ctrl+D to save and exit.

Example:

cat > myfile.txt
Now you can type your text directly. After typing, press Ctrl+D to save it to the file.

3. Using a Text Editor like nano
If you need to edit a file more interactively, you can use a text editor like nano:

nano filename.txt

This will open the file filename.txt in the nano editor. You can start typing right away, and when you're done, press Ctrl+X, then Y to confirm changes, and Enter to save.

4. Using printf Command
The printf command is similar to echo, but it gives you more control over the output formatting.

Example:

printf "Hello, this is a formatted line\n" > myfile.txt
This will create the file myfile.txt and write the formatted text into it.

Summary of Commands:
Write to a file (overwrite):

echo "text" > filename.txt
Append to a file:

echo "text" >> filename.txt
Write multiple lines interactively:

cat > filename.txt
(Press Ctrl+D when done)

Use a text editor:

nano filename.txt
Let me know if you'd like more details on any of these methods!

You said:
lets start with program 1
ChatGPT said:
Awesome! Let's begin with Program 1: Exploring Basic Linux Commands – Part 1.

We'll cover the following commands in this experiment:

mkdir – create directories

cd (instead of chdir, which is not commonly used) – change directories

cat – view contents of a file

ls – list files/directories

chmod – change file permissions

☑ Step-by-Step Instructions (Use these directly in your Replit Bash terminal)
◇ 1. mkdir – Make Directory

mkdir mydir
This creates a directory named mydir.

◇ 2. cd – Change Directory

cd mydir
You are now inside the mydir directory.

◇ 3. cat – Create and View File Contents
Let’s create a file using cat:

cat > hello.txt
Now type something like:

This is a sample file.
Learning Linux is fun!
Then press Ctrl+D to save and exit.

To view the file:

cat hello.txt
◇ 4. ls – List Files

ls
You should see hello.txt listed here.

For more details:

ls -l
◇ 5. chmod – Change File Permissions
Check current permissions:

ls -l hello.txt
Now change permissions to make it executable:

bash
Copy
Edit
chmod +x hello.txt
Verify again:

ls -l hello.txt

You'll notice the x permission added.

☑ Done with Program 1!
Let me know once you've tried all the steps — or if you want to write this up as a practical report format with explanations and outputs. Ready for Program 2 when you are!

You said:
what is the general syntax of chmod command explain
ChatGPT said:
Great question! Let's break down the general syntax of the chmod command and explain how it works.

🔧 General Syntax of chmod

chmod [options] mode filename
chmod: stands for change mode

mode: specifies the permission changes (who and what)

filename: the file or directory you want to modify

📌 There are 2 main ways to use chmod:
🅰 Symbolic Mode (more readable for beginners)
◇ Syntax:

chmod [who][operator][permission] filename
◇ Who:

Symbol	Meaning
u	user (owner)
g	group
o	others
a	all (u+g+o)

◇ Operator:

Symbol	Meaning
+	Add permission
-	Remove permission
=	Set exact permission

◇ Permission Types:

Symbol	Meaning
r	read
w	write
x	execute

☑ Examples:

bash
Copy
Edit
```
chmod u+x script.sh     # Give execute permission to the user (owner)
chmod g-w myfile.txt    # Remove write permission from group
chmod a+r file.txt      # Give read permission to everyone
chmod u=rw file.txt     # Set user permission to read and write only
```
🅱 Numeric (Octal) Mode
Permissions are represented by numbers:

Permission	Binary	Octal
---	000	0
--x	001	1
-w-	010	2
-wx	011	3
r--	100	4
r-x	101	5
rw-	110	6
rwx	111	7

So permission is set using three digits:
[user][group][others]

☑ Examples:

```
chmod 755 script.sh     # rwxr-xr-x (User: all, Group: read+exec, Others:
read+exec)
chmod 644 file.txt      # rw-r--r-- (User: read+write, Group: read, Others:
read)
chmod 700 secret.sh     # rwx------ (Only user has all permissions)
```
📝 Summary

Method	Syntax Example	Meaning
Symbolic	chmod u+x file.sh	Add execute for user
Numeric	chmod 755 file.sh	User: all, Group/Others: read+exec

Let me know if you want some fun exercises to try with chmod 😄

You said:
next program
ChatGPT said:
Nice! Let’s move on to Program 2, which explores more basic Linux commands.

☑ Program 2: Explore basic Linux commands like pwd, cd, mv, cp, rm
Here’s how each command works with examples you can run in Replit Bash:

◇ 1. pwd – Print Working Directory

pwd
What it does: Shows the full path of your current directory.

☑ Try this in your terminal and observe the output.

◇ 2. cd – Change Directory
We used this earlier too.

```
mkdir folder1
cd folder1
```
What it does: Changes to folder1 directory.

To go back up one level:

```
cd ..
```
◇ 3. mv – Move or Rename Files
Create a file first:

```
echo "Test file for mv" > oldname.txt
```
Rename the file:

```
mv oldname.txt newname.txt
```
Move a file to another directory:

```
mkdir target
mv newname.txt target/
```
◇ 4. cp – Copy Files
Make sure you have a file (or bring one back from the above step):

```
cp target/newname.txt copyfile.txt
```
This creates a copy named copyfile.txt in the current directory.

To copy to another directory:

```
cp copyfile.txt target/
```
◇ 5. rm – Remove Files or Directories
Warning: This permanently deletes files or folders!

Remove a file:

```
rm copyfile.txt
```
Remove an empty folder:

```
rmdir folder1
```

Remove a folder and everything in it:

```
rm -r target
```

Program 3a: Display OS version, release number, kernel version

🔧 1. cat /etc/os-release
Purpose: Shows general information about your operating system.

What it displays: OS name, version, ID, and other metadata.

☑ Very useful to know what distro (e.g., Ubuntu, Debian) you're on.

Example Output:

makefile
Copy
Edit
NAME="Ubuntu"
VERSION="20.04.6 LTS (Focal Fossa)"
ID=ubuntu
...
🔧 2. lsb_release -a
Purpose: Prints Linux Standard Base (LSB) information.

Gives more detailed info about:

Distributor ID

Release version

Codename (e.g., focal, jammy)

Example Output:

```
Distributor ID: Ubuntu
Description:    Ubuntu 20.04.6 LTS
Release:        20.04
Codename:       focal
```

Note: If you're using Replit and this command gives "command not found", it's because Replit doesn’t include the lsb_release tool by default. You can skip it there.

🔧 3. hostnamectl
Purpose: Shows details like:

Hostname (machine name)

Operating system

Kernel

Architecture (e.g., x86_64)

Example Output:

yaml
Copy
Edit
```
   Static hostname: my-pc
         Icon name: computer-vm
           Chassis: vm
        Machine ID: ...
           Boot ID: ...
  Operating System: Ubuntu 20.04.6 LTS
            Kernel: Linux 5.15.0-91-generic
      Architecture: x86-64
```
☑ Great for checking system identity and specs.

🔧 4. uname -r
Purpose: Shows your kernel version only.

It’s quick and often used in scripts.

Example Output:

Copy
Edit
5.15.0-91-generic
The kernel version tells you what version of the Linux kernel your system is running on.

📝 Summary Table:

Command	What it Shows
cat /etc/os-release	OS name and version (text-based info)
lsb_release -a	LSB-compliant release info (if installed)
hostnamectl	Hostname, OS, Kernel, Architecture
uname -r	Kernel version only

☑ Program 3b: Shell Script to Display Top 10 Processes by Memory Usage
◇ Shell Script Code

```
#!/bin/sh

ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%mem | head -n 11
```
🤔 Explanation of the Command

Part of Command	What it Does
ps	Lists currently running processes
-e	Show processes for all users
-o pid,ppid,cmd,%mem,%cpu	Output format: Process ID, Parent PID, Command, Memory%, CPU%

--sort=-%mem   Sort processes by descending memory usage
head -n 11   Show the top 10 (plus header line)
💡 head -n 11 is used because the ps command outputs a header as the first line, followed by 10 processes.

▶ How to Use on Replit Bash
Create the script file:

touch topmem.sh
Edit it:

nano topmem.sh
Paste the script, save (Ctrl + O, Enter) and exit (Ctrl + X).

Make it executable:

bash
Copy
Edit
chmod +x topmem.sh
Run the script:

bash
Copy
Edit
./topmem.sh
☑ Expected Output Sample
swift
Copy
Edit
```
  PID  PPID CMD                         %MEM %CPU
  135     1 /usr/lib/firefox/firefox     8.2  3.1
  202   135 /usr/lib/firefox/plugin      6.1  1.7
  ...
```

Program 4: Producer-Consumer Problem in C 🥷
This is a classic inter-process communication (IPC) problem that demonstrates how processes can synchronize with each other using semaphores. The idea is that the Producer generates data and the Consumer consumes it, with a buffer in between to hold the data.

◇ Producer-Consumer Problem in C using Semaphores
Objective: Use semaphores to manage the synchronization between a producer and a consumer in a buffer. The producer will produce an item, and the consumer will consume it, ensuring that:

The consumer doesn't try to consume from an empty buffer.

The producer doesn’t try to add to a full buffer.

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <unistd.h>         // For sleep()
#include <pthread.h>        // For pthread_create(), pthread_join(), etc.
#include <semaphore.h>      // For sem_t, sem_init(), sem_wait(), sem_post()

#define SIZE 5              // Size of the buffer
int buffer[SIZE];           // Shared buffer
int in = 0;                 // Points to the next empty slot for producer
int out = 0;                // Points to the next full slot for consumer

sem_t empty;                // Semaphore to count empty slots
sem_t full;                 // Semaphore to count full slots
pthread_mutex_t mutex;      // Mutex for mutual exclusion of buffer access

// Producer function
void* producer(void* arg) {
    int item;
    for (int i = 0; i < 10; i++) {
        item = rand() % 100; // Produce a random item

        sem_wait(&empty);             // Wait if buffer is full (decrement empty
slots)
        pthread_mutex_lock(&mutex);   // Lock the buffer (critical section)

        buffer[in] = item;            // Put item in buffer
        printf("Producer produced: %d at index %d\n", item, in);
        in = (in + 1) % SIZE;         // Circular increment

        pthread_mutex_unlock(&mutex); // Unlock the buffer
        sem_post(&full);              // Signal that buffer has one more full
slot

        sleep(1); // Sleep to simulate time taken to produce
    }
    return NULL;
}

// Consumer function
void* consumer(void* arg) {
    int item;
    for (int i = 0; i < 10; i++) {
        sem_wait(&full);              // Wait if buffer is empty (decrement full
slots)
        pthread_mutex_lock(&mutex);   // Lock the buffer (critical section)

        item = buffer[out];           // Remove item from buffer
        printf("Consumer consumed: %d from index %d\n", item, out);
        out = (out + 1) % SIZE;       // Circular increment

        pthread_mutex_unlock(&mutex); // Unlock the buffer
        sem_post(&empty);             // Signal that buffer has one more empty
slot

        sleep(1); // Sleep to simulate time taken to consume
```

```
    }
    return NULL;
}

int main() {
    pthread_t prodThread, consThread; // Thread identifiers

    // Initialize semaphores
    sem_init(&empty, 0, SIZE); // Initially buffer is empty, so SIZE empty slots
    sem_init(&full, 0, 0);     // Initially buffer has 0 full slots

    // Initialize mutex
    pthread_mutex_init(&mutex, NULL);

    // Create producer and consumer threads
    pthread_create(&prodThread, NULL, producer, NULL);
    pthread_create(&consThread, NULL, consumer, NULL);

    // Wait for threads to complete
    pthread_join(prodThread, NULL);
    pthread_join(consThread, NULL);

    // Destroy semaphores and mutex
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

5. Write shell scripts to do the following:
a. Display processes with highest memory usage.

```
#!/bin/sh
Top –b –o +%MEM | head -17
```

b.Display current logged in user and log name.

3a

```
#!/bin/bash

# Display top processes with highest memory usage
# 'top' in batch mode, sorted by %MEM, and print top 10 lines

top -b -n 1 | head -n 17
```

p -b -n 1 → Run top once in batch mode (no interactive screen).

head -n 17 → The first 7 lines are headers, so you see top 10 processes.

3b

b) Display current logged-in user and log name

Script: userinfo.sh

```
#!/bin/bash

# This script displays the current logged in user and their login name.

echo "Logged-in user: $(whoami)"
echo "Login name: $LOGNAME"
```

Explanation:

whoami → returns the current username.

$LOGNAME → is an environment variable that stores your login name.

📜 Exp 6:
Create a child process in Linux using the fork system call.
From the child process, obtain the process ID of both child and parent by using getpid() and getppid() system calls.

✸ Theory:
What is a Process?
A process is a program that is currently being executed.

Each process has:

a unique ID (called PID – Process ID)

a Parent Process ID (PPID).

What is fork()?
fork() is a system call used to create a new process.

The new process created is called a child process.

The child process is an exact copy of the parent process except for the returned value of fork().

What happens after fork()?
fork() returns:

Zero (0) to the child process.

Positive value (PID of child) to the parent process.

Negative value if the creation of child process fails.

Important Functions:

Function Purpose
getpid() Returns the Process ID of the currently running process.
getppid() Returns the Parent Process ID of the currently running process.

🛠 Program Code (With Detailed Comments):

```
#include <stdio.h>      // For printf()
#include <unistd.h>     // For fork(), getpid(), getppid()

int main() {
    pid_t pid;          // 'pid_t' is a data type for process IDs

    pid = fork();       // Create a child process

    if (pid < 0) {
        // fork() failed (child process not created)
        printf("Failed to create child process.\n");
    }
    else if (pid == 0) {
        // This block is executed by the child process
        printf("\nThis is the Child Process.\n");
        printf("Child Process ID (PID): %d\n", getpid());      // Child's PID
        printf("Parent Process ID (PPID): %d\n", getppid());   // Child's Parent
PID
    }
    else {
        // This block is executed by the parent process
        printf("\nThis is the Parent Process.\n");
        printf("Parent Process ID (PID): %d\n", getpid());     // Parent's PID
        printf("Child Process ID (PID): %d\n", pid);           // Child's PID
(returned by fork())
    }

    return 0;
}
```

📖 Experiment 7

Explore wait() and waitpid() before termination of a process.

🟢 What is wait()?

wait() is used by a parent process to wait until one of its child processes finishes (terminates).

When a child process ends, the parent collects its exit status using wait().

It helps to clean up the resources used by the child and avoid zombie processes.

Syntax:

```
pid_t wait(int *status);
```

If you don't care about exit status, you can pass NULL as the argument.

What is waitpid()?

waitpid() is more advanced than wait().

It allows the parent to wait for a specific child process (using child's PID).

Syntax:

```
pid_t waitpid(pid_t pid, int *status, int options);
```

Useful if the parent has multiple children and wants to control which one to wait for.

Command	Purpose
wait()	Wait for any child process to terminate.
waitpid(pid, ...)	Wait for a specific child process by PID.

Simple Program Demonstrating fork(), wait()

```
#include <stdio.h>      // printf()
#include <sys/types.h>  // pid_t
#include <sys/wait.h>   // wait(), waitpid()
#include <unistd.h>     // fork()

int main() {
    pid_t pid;

    pid = fork(); // Create child process

    if (pid < 0) {
        printf("Fork failed.\n");
    }
    else if (pid == 0) {
        // Child Process
        printf("Child: My PID is %d\n", getpid());
        printf("Child: I am sleeping for 5 seconds.\n");
        sleep(5);  // Child sleeps
        printf("Child: I am done.\n");
    }
    else {
        // Parent Process
        printf("Parent: Waiting for child to complete.\n");
        wait(NULL); // Parent waits for child to finish
        printf("Parent: Child has completed.\n");
    }

    return 0;
}
```

🛠 Program using waitpid()

```
#include <stdio.h>      // printf()
#include <sys/types.h>  // pid_t
#include <sys/wait.h>   // wait(), waitpid()
#include <unistd.h>     // fork()
```

```
int main() {
    pid_t pid, wpid;
    int status;

    pid = fork(); // Create child process

    if (pid < 0) {
        printf("Fork failed.\n");
    }
    else if (pid == 0) {
        // Child Process
        printf("Child: My PID is %d\n", getpid());
        printf("Child: Sleeping for 5 seconds.\n");
        sleep(5);
        printf("Child: Exiting now.\n");
    }
    else {
        // Parent Process
        printf("Parent: Waiting for child with PID = %d\n", pid);
        wpid = waitpid(pid, &status, 0); // Wait for the specific child

        if (wpid == -1) {
            printf("Parent: Error while waiting for child.\n");
        } else {
            printf("Parent: Child with PID %d has finished.\n", pid);
            // Optionally check if child exited normally
            if (WIFEXITED(status)) {
                printf("Parent: Child exited with status %d.\n",
WEXITSTATUS(status));
            }
        }
    }

    return 0;
}
```

Let's go! 🚀
Moving to*Experiment 9;

📄 Experiment 9:
> **Write a program in C to do disk scheduling - FCFS (First Come First Serve).**

📖 Quick Theory:

🟢 What is Disk Scheduling?

FCFS Disk Scheduling - Theory
FCFS (First-Come, First-Served) Disk Scheduling is the simplest disk scheduling algorithm.

In FCFS:

Requests are attended in the order in which they arrive.

The disk head moves from the current position to the requested track without reordering the queue.

It is similar to standing in a line: whoever comes first is served first.

How it Works:

The disk head services each I/O request in the sequence they arrive.

There is no optimization of seek time (travel distance of the disk arm).

Advantages:

Simple to understand and implement.

Fair to all processes – no starvation.

Disadvantages:

High seek time if requests are far apart.

Does not guarantee optimal performance compared to more advanced algorithms like SSTF (Shortest Seek Time First).

Important Term:

Seek Time: The time taken by the disk arm to move to the track where data is located.

Total Seek Time: The sum of the distances moved by the head to fulfill all the requests.

🛠 **C Program for Disk Scheduling - FCFS**

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, i, initial;
    int total_movement = 0;

    printf("Enter the number of disk requests: ");
    scanf("%d", &n);
```

```
    int request[n];

    printf("Enter the disk request sequence:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &request[i]);
    }

    printf("Enter the initial head position: ");
    scanf("%d", &initial);

    printf("\nDisk Head Movement:\n");
    printf("%d", initial);

    for(i = 0; i < n; i++) {
        total_movement += abs(request[i] - initial);
        initial = request[i];
        printf(" -> %d", request[i]);
    }

    printf("\n\nTotal head movement = %d cylinders\n", total_movement);

    return 0;
}
```

🎯 In One Line:
> **FCFS Disk Scheduling** simply **serves requests in order**, calculating the distance between each movement.

Would you also like a **small flow diagram** 📈 for how head moves in FCFS (very helpful for viva)?
It will take just 1 min to show! 🚀🎯

Program 8 : Bankers Algorithm

The Banker's Algorithm is a deadlock avoidance algorithm used in operating systems.
It tests safe states to allocate resources to processes without leading to a deadlock.

It is called the Banker's Algorithm because it is similar to a bank managing customer loans – the bank will not grant a loan if it leads the system (bank) into an unsafe (unstable) state.

Key Concepts:

Each process must declare the maximum number of resources it may need.

Resources are only allocated if doing so does not lead the system into an unsafe

state.

The system checks for a safe sequence of process execution where all processes can finish without deadlock.

Important Terms:

Allocation: Resources currently allocated to a process.

Max: Maximum resources a process may request.

Available: Resources currently available in the system.

Need: Resources still needed by a process = Max - Allocation.

The algorithm continuously checks if resources can be allocated safely and updates the system state.

Formatted & Commented Code for Banker's Algorithm

```
#include <stdio.h>

int main()
{
    // P0, P1, P2, P3, P4 are the Process names

    int n, m, i, j, k;
    n = 5; // Number of processes
    m = 3; // Number of resources (resource types)

    // Allocation Matrix: resources currently allocated to each process
    int alloc[5][3] = {
        { 0, 1, 0 }, // P0
        { 2, 0, 0 }, // P1
        { 3, 0, 2 }, // P2
        { 2, 1, 1 }, // P3
        { 0, 0, 2 }  // P4
    };

    // Maximum demand of each process
    int max[5][3] = {
        { 7, 5, 3 }, // P0
        { 3, 2, 2 }, // P1
        { 9, 0, 2 }, // P2
        { 2, 2, 2 }, // P3
        { 4, 3, 3 }  // P4
    };

    // Available Resources: initially available amount for each resource type
    int avail[3] = { 3, 3, 2 };

    int f[n], ans[n], ind = 0; // f[i] will indicate if process i is finished
```

```
    // Initialize all processes as not finished
    for (k = 0; k < n; k++) {
        f[k] = 0;
    }

    int need[n][m]; // To store the NEED matrix

    // Calculate Need Matrix = Max - Allocation
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
    }

    int y = 0;

    // Main loop to find the safe sequence
    for (k = 0; k < 5; k++) // repeat 5 times to ensure all processes are
checked
    {
        for (i = 0; i < n; i++)
        {
            if (f[i] == 0) // If process is not yet finished
            {
                int flag = 0; // Assume process can execute

                for (j = 0; j < m; j++)
                {
                    if (need[i][j] > avail[j]) {
                        flag = 1; // Cannot execute if need > available
                        break;
                    }
                }

                if (flag == 0) // If all needs are satisfied
                {
                    ans[ind++] = i; // Save the process number in the answer
array

                    // Now pretend that process i finishes and releases its
resources
                    for (y = 0; y < m; y++)
                        avail[y] += alloc[i][y];

                    f[i] = 1; // Mark process as finished
                }
            }
        }
    }

    // Print the Safe Sequence
    printf("Following is the SAFE Sequence:\n");
    for (i = 0; i < n - 1; i++)
```

```
        printf(" P%d ->", ans[i]);
    printf(" P%d\n", ans[n - 1]);

    return 0;
}
```